

Alumni Influencers API

Endpoint Documentation

Version 1.0.0

Developer: Phantasmagoria Ltd

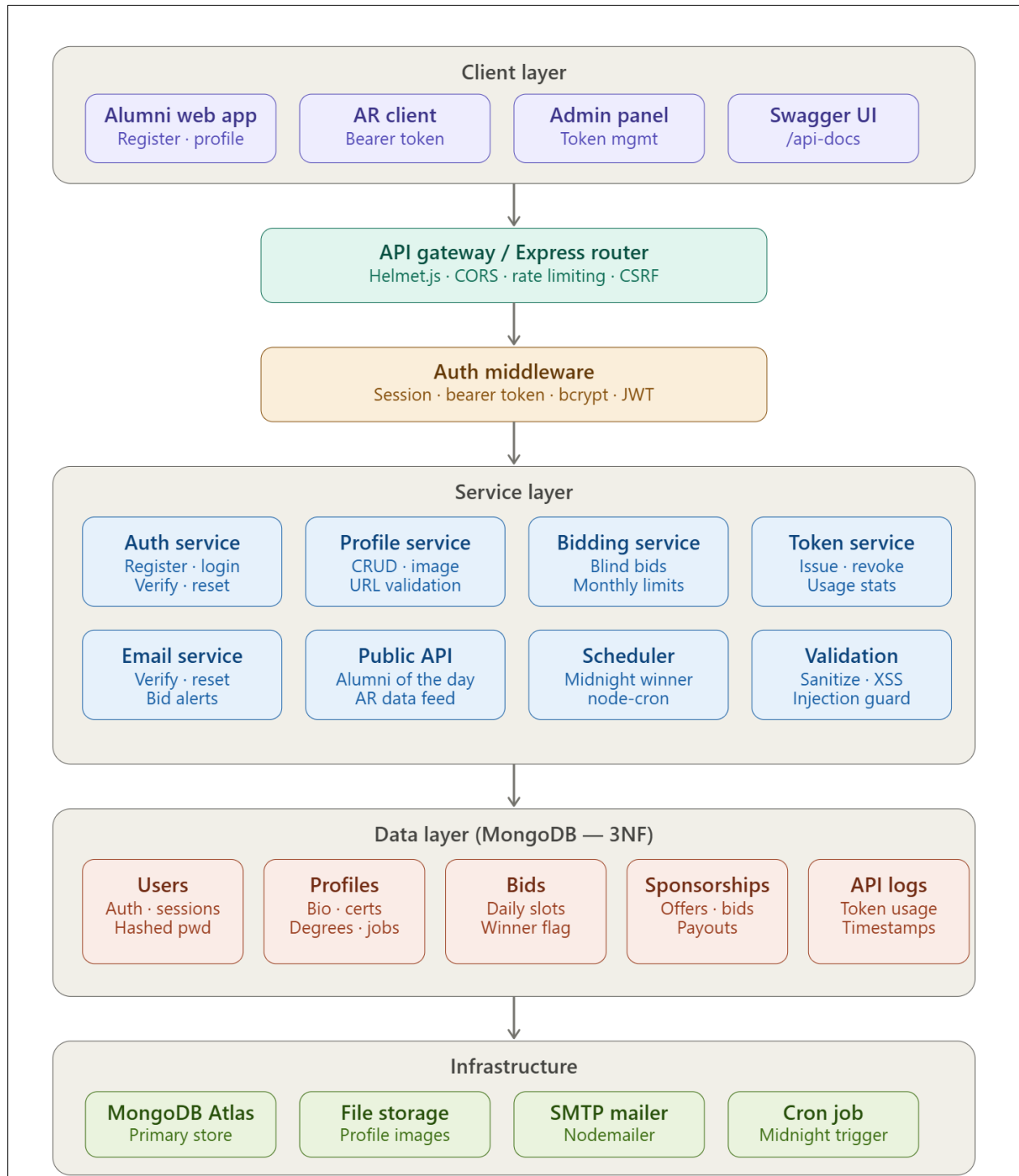
Base URL: <http://localhost:3000/api>

1. System Overview	4
2. Authentication & Security	5
2.1 Bearer JWT Token	5
2.2 API Key	5
2.3 Email Verification.....	6
2. Authentication Endpoints.....	6
2.1 Register	6
Validation Rules.....	6
Request Body	6
Response — 200 OK	6
Error Responses	7
2.2 Login	7
Request Body	7
Response — 200 OK	7
Error Responses	7
3. Profile Management.....	8
3.1 Get My Profile	8
Authentication.....	8
Response — 200 OK	8
3.2 Create / Update Profile.....	9
Authentication.....	9
Supported Fields	9
Degree Object	9
3.3 Upload Profile Photo.....	9
Authentication.....	9

Request.....	9
Constraints	10
4. Bidding System.....	10
4.1 Place / Increase Bid.....	10
Authentication.....	10
Request Body	10
Request Parameters	10
Response — Winning Bid (200 OK)	10
Response — Outbid (200 OK).....	11
4.2 Check Bid Status.....	11
Authentication.....	11
Response Fields	11
4.3 Bid History.....	11
Authentication.....	11
5. Public Client API (AR Client)	12
5.1 Get Alumni of the Day.....	12
Authentication.....	12
Response — 200 OK	12
Error Responses	12
6. Admin Management.....	13
6.1 Generate API Key	13
Authentication.....	13
Request Body	13
Response — 201 Created.....	13
6.2 List All API Keys.....	14
Authentication.....	14
Response — 200 OK	14
6.3 Deactivate (Revoke) API Key.....	14
Authentication.....	14
Path Parameters.....	14
Response — 200 OK	14
6.4 Activate API Key	15
Authentication.....	15
Path Parameters.....	15
Response — 200 OK	15
6.5 Get API Key Usage Stats	15

Authentication.....	15
Path Parameters.....	15
Response — 200 OK	15
6.6 List Global Usage Logs	16
Authentication.....	16
Response — 200 OK	16
7. Standard Error Codes	16
8. Endpoint Quick Reference	17

1. System Overview



Client layer — four distinct consumer types: the alumni web app (for registration and profile management), the AR client (the external consumer that uses bearer tokens), an admin panel for token management and usage stats, and Swagger UI for the public developer API at /api-docs.

API gateway — all requests pass through Express middleware enforcing Helmet.js security headers, CORS configuration, rate limiting on sensitive endpoints, and CSRF protection. This maps directly to the Security Implementation section.

Auth middleware — sits between the gateway and the services, handling session validation, bearer token verification, bcrypt password checks, and JWT processing. Every service call is authenticated before it proceeds.

Service layer — the business logic split across eight focused services. Notably the Bidding service encapsulates blind-bidding logic (no highest bid exposed), monthly limit tracking, and the Scheduler service runs a node-cron job at midnight to automatically select the winner and mark their profile.

Data layer — five MongoDB collections are structured to 3NF as required. The Bids collection carries the winner flag used by the public API. API Logs tracks token usage, endpoints accessed, and timestamps for the admin stats view.

Infrastructure — the supporting services including MongoDB for persistence, a file store for profile image uploads, Nodemailer for the email verification/reset/notification flows, and the cron trigger for automated winner selection.

2. Authentication & Security

The API uses two types of authentication depending on the endpoint:

2.1 Bearer JWT Token

Most protected endpoints require a JSON Web Token (JWT) passed in the Authorization header.

Obtain via: POST /auth/login

Header: Authorization: Bearer <your_token>

2.2 API Key

Used specifically for the Public Client API (e.g., AR application). API keys are generated by administrators.

Obtain via: POST /admin/apikeys (Admin only)

Header: x-api-key: <your_api_key>

2.3 Email Verification

All newly registered accounts must verify their email address before being permitted to log in. Verification emails are sent automatically upon registration and must originate from a recognised university domain.

2. Authentication Endpoints

These endpoints handle user registration, login, and account verification. No authentication token is required to access them.

Authentication			^
POST	/auth/register	Register a new alumni	▼
POST	/auth/login	Login and get JWT token	▼
GET	/auth/logout	Logout current user	🔒 ▼
GET	/auth/verifyemail/{token}	Verify email address	▼
POST	/auth/forgotpassword	Request a password reset email	▼
PUT	/auth/resetpassword/{token}	Reset password using the token from email	▼

2.1 Register

[POST] /auth/register

Creates a new user account. A verification email is automatically sent on success.

Validation Rules

- Email must be a recognised university address (.ac.uk or .edu domain)
- Password must contain at least one uppercase letter, one number, and one special character

Request Body

Field	Example	Required
email	john.smith@eastminster.ac.uk	Yes
password	Secure@Pass1!	Yes

Response — 200 OK

```
{
  "success": true,
  "message": "Registration successful. Please check your email to verify your account."
}
```

Error Responses

Status	Description	Notes
400	Invalid email domain or password format	-
409	Email address already registered	-

2.2 Login

[POST] /auth/login

Authenticates a verified user and returns a JWT bearer token.

Request Body

Field	Example	Required
email	john.smith@eastminster.ac.uk	Yes
password	Secure@Pass1!	Yes

Response — 200 OK

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error Responses

Status	Description	Notes
401	Invalid credentials	-
403	Account not yet verified	-

3. Profile Management

All profile endpoints are protected and require a valid Bearer JWT token. These endpoints allow users to view and update their public alumni profile, including degrees, employment history, and profile photo.

Profile			^
GET	/profile	Get all alumni profiles	 ▼
POST	/profile	Create or update the current user's profile	 ▼
GET	/profile/me	Get the current user's own profile	 ▼
GET	/profile/user/{user_id}	Get a specific alumni profile by user ID	▼
PUT	/profile/uploadphoto	Upload a profile photo	 ▼

3.1 Get My Profile

[GET] /profile/me

Returns the full profile and appearance history of the currently authenticated user.

Authentication

Bearer Token required.

Response — 200 OK

```
{
  "success": true,
  "data": {
    "firstName": "John",
    "lastName": "Smith",
    "bio": "Software Engineer specialized in AI.",
    "profileImage": "http://localhost:3000/uploads/file-123.jpg",
    "appearanceCount": 3,
    "bonusSlotAvailable": false,
    "degrees": [
      {
        "degreeTitle": "BSc Computer Science",
        "university": "University of Eastminster",
        "completionDate": "2022-06-15"
      }
    ]
  }
}
```


3.2 Create / Update Profile

[POST] /profile

Creates or updates the authenticated user's profile. Supports nested objects for degrees and employment history.

Authentication

Bearer Token required.

Supported Fields

Field	Example	Required
firstName	John	Yes
lastName	Smith	Yes
bio	Software Engineer specialized in AI.	No
degrees	Array of degree objects	No
employment	Array of employment objects	No

Degree Object

Field	Example	Required
degreeTitle	BSc Computer Science	Yes
university	University of Eastminster	Yes
completionDate	2022-06-15	Yes

3.3 Upload Profile Photo

[PUT] /profile/uploadphoto

Uploads or replaces the authenticated user's profile image. Uses multipart/form-data encoding.

Authentication

Bearer Token required.

Request

Field	Accepted Types	Required
file	image/jpeg, image/png	Yes

Constraints

- Maximum file size: 1 MB
- Field name must be: file
- Content-Type header must be: multipart/form-data

4. Bidding System

The bidding system allows authenticated alumni to compete for the coveted "Alumni of the Day" spotlight slot. Bidding is blind — participants cannot see competing bids. The highest bidder at the daily cut-off wins the slot for the following day.

Bidding			^
POST	/bids	Place or update a bid for tomorrow's featured slot	🔒 ▼
GET	/bids/status	Get current bid status for tomorrow's slot	🔒 ▼
GET	/bids/history	Get all past bids for the current user	🔒 ▼

4.1 Place / Increase Bid

[POST] /bids

Places a new bid or increases an existing bid for tomorrow's Alumni of the Day slot. Each user may only hold one active bid at a time; submitting a higher amount replaces the previous bid.

Authentication

Bearer Token required.

Request Body

```
{ "amount": 250.00 }
```

Request Parameters

Field	Example	Required
amount	250.00	Yes

Response — Winning Bid (200 OK)

```
{
  "success": true,
  "data": {
    "bidAmount": 250.0,
    "currency": "GBP",
  }
}
```

```
"bidStatus": "Winning",
"message": "You are currently the highest bidder for tomorrow's slot!",
"remainingWinsThisMonth": 2
}
}
```

Response — Outbid (200 OK)

```
{
  "success": true,
  "data": {
    "bidAmount": 250.0,
    "currency": "GBP",
    "bidStatus": "Outbid",
    "message": "Your bid has been placed but you are currently outbid.",
    "remainingWinsThisMonth": 2
  }
}
```

4.2 Check Bid Status

[GET] /bids/status

Returns the authenticated user's current bid status for tomorrow's slot without placing or modifying a bid.

Authentication

Bearer Token required.

Response Fields

Field	Example	Description
bidStatus	Winning / Outbid / NoBid	Current position in the blind auction
bidAmount	250.00	User's current bid in GBP
remainingWinsThisMonth	2	How many more wins are allowed this month

4.3 Bid History

[GET] /bids/history

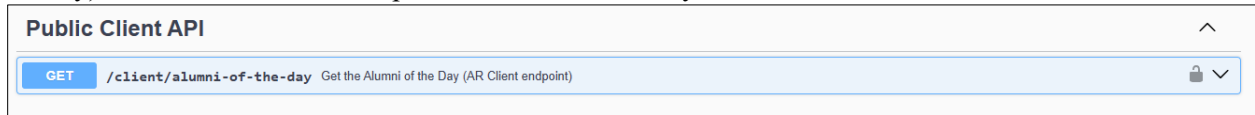
Returns a full list of all previous bids placed by the authenticated user, including outcomes.

Authentication

Bearer Token required.

5. Public Client API (AR Client)

These endpoints are designed for consumption by external client applications such as the AR (Augmented Reality) kiosk. Authentication is performed via an API key rather than a user JWT token.



5.1 Get Alumni of the Day

[GET] /client/alumni-of-the-day

Returns the current day's winning alumnus profile for display in client applications.

Authentication

API Key required in request header:

```
x-api-key: <your_api_key>
```

Response — 200 OK

```
{
  "success": true,
  "data": {
    "firstName": "John",
    "lastName": "Smith",
    "bio": "Software Engineer specialized in AI.",
    "profileImage": "http://localhost:3000/uploads/file-123.jpg",
    "degrees": [
      {
        "degreeTitle": "BSc Computer Science",
        "university": "University of Eastminster",
        "completionDate": "2022-06-15"
      }
    ]
  }
}
```

Error Responses

Status	Description	Notes
401	Missing or invalid API key	-
404	No alumni of the day found for today	-

6. Admin Management

Admin endpoints are restricted to users with the administrator role. These endpoints allow management of API keys issued to client applications and provide usage analytics.

Admin — API Keys			^
POST	/admin/apikeys	Generate a new API key (Admin only)	🔒 ▼
GET	/admin/apikeys	List all API keys (Admin only)	🔒 ▼
GET	/admin/apikeys/{id}/stats	Get usage statistics for an API key (Admin only)	🔒 ▼
DELETE	/admin/apikeys/{id}	Revoke an API key (Admin only)	🔒 ▼

6.1 Generate API Key

[POST] /admin/apikeys

Generates a new API key that can be issued to a client application (e.g., AR kiosk).

Authentication

Bearer Token required (Admin role).

Request Body

Field	Example	Required
label	AR Kiosk - Main Hall	Yes
expiresAt	2026-12-31	No

Response — 201 Created

```
{
  "success": true,
  "data": {
    "id": "key_abc123",
    "apiKey": "pk_live_XXXXXXXXXXXXXXXX",
    "label": "AR Kiosk - Main Hall",
    "createdAt": "2025-01-15T10:30:00Z"
  }
}
```

6.2 List All API Keys

[GET] /admin/apikeys

Returns a list of all generated API keys and their current activation status.

Authentication

Bearer Token required (Admin role).

Response — 200 OK

```
{
  "success": true,
  "data": [
    {
      "_id": "69a2ba49dc1074d860c1e975",
      "name": "AR Mobile Client",
      "isActive": true,
      "createdAt": "2026-02-28T09:00:00.000Z"
    }
  ]
}
```

6.3 Deactivate (Revoke) API Key

[DELETE] /admin/apikeys/:id

Immediately disables an API key. Any subsequent requests using this key will be rejected.

Authentication

Bearer Token required (Admin role).

Path Parameters

Parameter	Example	Required
:id	69a2ba49dc1074d860c1e975	Yes

Response — 200 OK

```
{
  "success": true,
  "message": "API Key revoked"
}
```

6.4 Activate API Key

[PUT] /admin/apikeys/:id/activate

Re-enables a previously revoked API key, restoring access for the associated client application.

Authentication

Bearer Token required (Admin role).

Path Parameters

Parameter	Example	Required
:id	69a2ba49dc1074d860c1e975	Yes

Response — 200 OK

```
{
  "success": true,
  "message": "API Key activated"
}
```

6.5 Get API Key Usage Stats

[GET] /admin/apikeys/:id/stats

Returns hit counts, timestamps, and IP addresses for a specific API key.

Authentication

Bearer Token required (Admin role).

Path Parameters

Parameter	Example	Required
:id	69a2ba49dc1074d860c1e975	Yes

Response — 200 OK

```
{
  "success": true,
  "data": {
    "name": "AR Client v1",
    "usageCount": 142,
    "usageStats": [
      {
        "date": "2026-02-28T10:00:00.000Z",
        "endpoint": "/api/client/alumni-of-the-day",
        "method": "GET"
      }
    ]
  }
}
```

```
}  
]  
}  
}
```

6.6 List Global Usage Logs

[GET] /admin/logs

Returns centralised usage logs for all API keys across the platform.

Authentication

Bearer Token required (Admin role).

Response — 200 OK

```
{  
  "success": true,  
  "count": 100,  
  "data": [  
    {  
      "apiKey": "69a2ba49dc1074d860c1e975",  
      "endpoint": "/api/client/alumni-of-the-day",  
      "method": "GET",  
      "timestamp": "2026-02-28T10:00:00.000Z"  
    }  
  ]  
}
```

7. Standard Error Codes

All endpoints return errors in a consistent JSON format. The table below summarises the common HTTP status codes used throughout the API.

Code	Status	Description
200	OK	Request completed successfully.
201	Created	Resource was created successfully.
400	Bad Request	Missing or invalid request data.
401	Unauthorized	Missing, expired, or invalid authentication credential.
403	Forbidden	Authenticated but insufficient permissions.
404	Not Found	The requested resource does not exist.
409	Conflict	Resources already exist (e.g. duplicate email).
413	Payload Too Large	Uploaded file exceeds the maximum size limit.

Code	Status	Description
500	Internal Server Error	An unexpected server-side error occurred.

8. Endpoint Quick Reference

The table below summarizes all available endpoints, their HTTP methods, authentication requirements, and a brief description.

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register new user account
POST	/auth/login	None	Login and receive JWT token
GET	/profile/me	Bearer Token	Get own profile + history
POST	/profile	Bearer Token	Create or update profile
PUT	/profile/uploadphoto	Bearer Token	Upload profile photo
POST	/bids	Bearer Token	Place or increase bid
GET	/bids/status	Bearer Token	Check current bid status
GET	/bids/history	Bearer Token	View all previous bids
GET	/client/alumni-of-the-day	API Key	Get today's winning alumni
POST	/admin/apikeys	Bearer Token (Admin)	Generate new API key
GET	/admin/apikeys	Bearer Token (Admin)	List all API keys
DELETE	/admin/apikeys/:id	Bearer Token (Admin)	Revoke an API key
PUT	/admin/apikeys/:id/activate	Bearer Token (Admin)	Activate a revoked key
GET	/admin/apikeys/:id/stats	Bearer Token (Admin)	View API key usage stats
GET	/admin/logs	Bearer Token (Admin)	List global usage logs